A VISUAL GUIDE

Crystal Graph Neural Networks

From crystal structure to a 554,054-material thermal-conductivity screen — a from-scratch reproduction and stress-test of PINK

Aizaz Khalid · reproduction of PINK, J. Mater. Inf. 2025, 5, 12

Why predict elastic moduli at all?

The bottleneck

Elastic moduli come from DFT calculations costing hours to days of compute – only ∼11,000 of the Materials Project's ∼150,000 entries have them.

Screening a large candidate space by DFT alone is not affordable.

Why they matter here

PINK screens for materials with ultralow *lattice thermal conductivity* κ_L, whose physics stage needs bulk modulus K and shear modulus G as inputs.

Predicting K and G in milliseconds turns an impossible screen into a routine one.

A CIF file says where the atoms are. It does not say how stiff the crystal is – that is what we are learning to predict.

Why not just a standard neural network?

A dense network expects a fixed-length vector; a CNN expects a fixed-size grid. Neither fits a crystal:

Variable size

A unit cell can hold 2 atoms or 200 – no fixed vector length to build a dense layer around.

No grid

Atoms sit at arbitrary 3D positions, not on the regular pixel grid a CNN's kernel slides across.

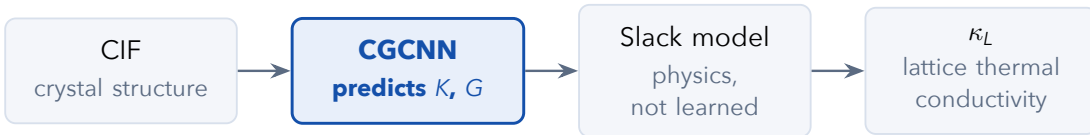
It repeats forever

A crystal is periodic – tiling space in three directions, not croppable to a fixed patch.

The fix: represent it as a *graph* – atoms as nodes, bonds as edges – which natively handles a variable number of objects in arbitrary positions. A graph neural network passes messages along the edges instead of convolving over pixels; CGCNN is exactly this, built for crystals.

Everything from here on is about the two things a graph needs: a node, and an edge.

Where this project fits inside PINK



The elastic moduli are the *only* learned quantity. Everything downstream of them is physics, not machine learning.

Parts Two through Five build the first arrow from scratch – CIF $\rightarrow (K, G)$ – including the network itself, not forked from the paper's code. Parts Six through Eight then carry that output through the physics stage, a real-data validation, and the actual 554,054-material screen.

Scope of this talk: the entire PINK pipeline, reproduced and independently stress-tested end to end – not just the machine-learning half.

PART TWO

From Crystal to Graph

Nodes, edges, and a worked example in real sodium chloride

Meet the crystal: rock-salt NaCl

Ordinary table salt. Its structure – **rock salt** – is one of the simplest in nature, and every number here comes from the actual CIF file used throughout this talk.

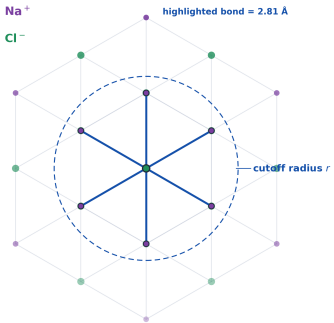
Cubic cell, side $a = 5.62$ Å
8 ions per cell: 4 Na^+, 4 Cl^-.
Every Na^+ sits surrounded by 6 Cl^- neighbours, all at exactly $a/2 = 2.81$ Å – the real crystal's coordination, not a simplification.

Fractional coordinates, from the CIF:

ion	*x*	*y*	*z*
Na^+	0	0	0
Na^+	0	0.5	0.5
Na^+	0.5	0	0.5
Na^+	0.5	0.5	0
Cl^-	0.5	0.5	0.5
Cl^-	0.5	0	0
Cl^-	0	0.5	0
Cl^-	0	0	0.5

A CIF is nothing more than a table like this one, plus the cell size and shape. Turning it into a graph is the next three slides.

Nodes, edges, and a cutoff radius



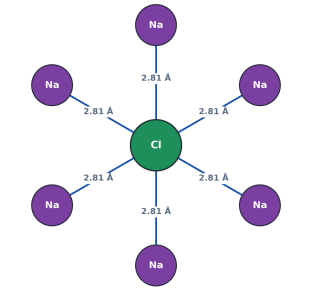
Nodes = atoms. Every ion in the faded, infinite lattice is one node.

Edges = bonds to neighbours *within a cutoff radius r* only (bold, $r \approx 2.9$ Å here).

For this Cl^-, that cutoff catches exactly its 6 nearest Na^+.

In the real pipeline, $r = 8$ Å and up to 12 neighbours are kept per atom – the idea is identical, just with more neighbours in view.

The abstraction: what the network sees



Strip away the projection, the perspective, the rest of the crystal fading into the background — and this is what is left: one small, labelled graph.

No 3D coordinates are given to the network directly. Only:

- which atom each node is
- which pairs of atoms are connected
- how far apart each connected pair is

This is not a simplification of what CGCNN sees — it *is* what CGCNN sees. A crystal, to this network, is exactly this object.

What's inside a node

Each node must carry *what element it is*. Rather than learning this from scratch, every atom is looked up in a fixed table.

`atom_init.json`: one 92-dimensional vector per element, built once from tabulated properties – group, period, electronegativity, covalent radius, valence count – one-hot binned.

These numbers are **fixed, never learned** – the starting description of "what element is this", identical for every atom of a given element in the dataset.

The network's first layer is a linear projection turning this fixed 92-dim description into the model's hidden width – the one place element identity becomes something *learned*.

Analogous to a lookup embedding table in NLP – except these starting vectors are physically meaningful, not randomly initialised.

Expanding the bond length

A bond has one obvious number: its length. Feeding that number in raw turns out to be a bad idea.

$$e_k(d) = \exp\left(-\frac{(d-\mu_k)^2}{\sigma^2}\right) \quad \mu_k = 0, 0.2, \ldots, 8 \text{ Å} \quad \sigma = 0.2 \text{ Å}$$

The problem with one number

A single scalar forces the network to learn a sharply non-linear response from scratch – hard to fit, slow to converge.

What the expansion buys

A 2.31 Å bond lights up the basis functions near 2.3, barely touching the one at 5.0 – smooth and easy to learn, not a cliff.

This is a radial basis expansion – the same trick used throughout atomistic machine learning, not something specific to CGCNN.

Assembling one training example

For a crystal of N atoms, the graph handed to the network is exactly three tensors:

Atom features

$N \times 92$

one fixed row per atom

Bond features

$N \times 12 \times 41$

Gaussian-expanded distance to each of up to 12 neighbours

Neighbour index

$N \times 12$

which row of the atom table each neighbour is

This triple, plus the true modulus in GPa, is one row of training data. The periodic neighbour search that builds it is the single slowest step in the whole pipeline – for 10,987 crystals it is computed once and cached.

Every crystal in this project, no matter how large or exotic, is reduced to exactly this shape before it ever reaches the network.

PART THREE

The Network Itself

How the graph turns into one number

In plain words, before the equations

"Each atom asks its neighbours: what are you, and how far away? Then it updates its own description based on the answers it hears."

That single round of asking-and-updating is called **message passing**, and it is the entire idea behind the network's convolution step. Do it once, and every atom's description reflects its immediate neighbours. Do it three times — as this network does — and every atom's description reflects everything happening within three bonds of it.

The next slide formalises exactly this sentence as an equation. Nothing new is being introduced — only made precise.

If the equations on the next slide feel abstract, come back to this sentence. That is all they are computing.

The gated convolution

For atom i with neighbours j, form the (self, neighbour, bond) triple for every edge, then update:

$$z_{ij} = v_i \oplus v_j \oplus e_{ij}$$

$$v_i \leftarrow v_i + \sum_j \sigma(z_{ij}W_f + b_f) \odot g(z_{ij}W_s + b_s)$$

$\oplus$ concatenation $\odot$ elementwise product σ sigmoid (the gate) g softplus (the message) v_i the running description of atom i

One shared linear layer processes every edge in the entire crystal identically. That sharing is what makes this a *convolution*: the same local rule applies everywhere, whether the crystal has 8 atoms or 200.

This one update, stacked three times, is the entire learned part of turning a graph into a description of the crystal.

Three design choices, and why

The gate $\sigma(\cdot)$

Decides, per bond, how much that neighbour contributes. A weak, distant bond can be down-weighted without being deleted from the graph.

Shared weights

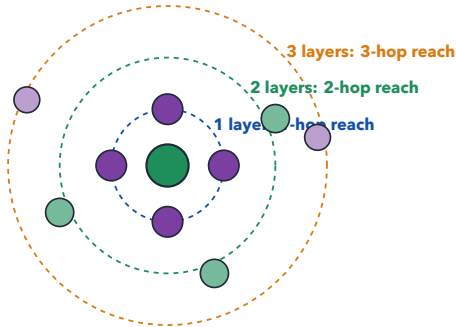
The same linear layer processes every edge. That is what makes this a convolution, and why it generalises across crystal sizes.

The residual, $v_i + (\cdot)$

Gradients flow straight through the stack, so early layers are not washed out by later ones as three layers compound.

Three convolution layers are used here, so every atom's final description reflects roughly a three-bond-hop neighbourhood.

Stacking layers grows the neighbourhood



Each additional layer lets information travel one more bond hop. After 3 layers, the centre atom's description is shaped by every atom within 3 bonds — with no global, whole-crystal operation ever computed.

More layers see further, at the cost of more computation and a risk of over-smoothing — which is why 3 is a deliberate choice, not a default.

From atoms to one number: pooling

$$v_c = \frac{1}{N}\sum_{i=1}^{N} v_i$$

Average, not sum, over every atom's final vector to get one description of the whole crystal.

A modulus is an **intensive** property – doubling the unit cell must not double the prediction.

A *sum* would make the output scale with however many atoms the CIF author happened to write down. The *mean* does not.

A small fully-connected head then turns that one 128-dimensional vector into a single number: the predicted modulus.

This is the one place the model has to actively resist a bias the data would otherwise hand it for free.

What we predict, and why its logarithm

Train on $\log_{10}$(modulus)

Moduli here span 1–575 GPa. Under plain MSE on raw values, a 40 GPa error on a stiff crystal produces $100\times$ more gradient than the same *relative* error on a soft one.

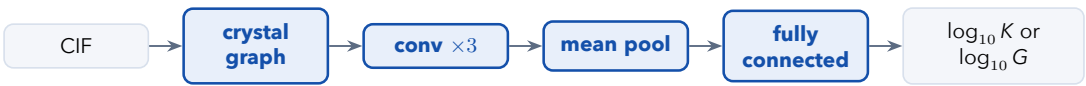
Exactly the wrong bias here

PINK screens for **ultralow** conductivity – the soft end of the range. A model that only cares about stiff crystals is worthless for this project's actual goal.

The logarithm converts "within a factor of x" into "within a fixed distance" – so soft and stiff crystals carry equal weight during training. Normalisation statistics (mean, std) come from the training split only, so no information about held-out crystals leaks in.

This single transform is why the model is not simply best at predicting hard, boring materials.

The full forward pass, start to finish



Every piece of this diagram has now been explained: how a crystal becomes a graph (Part 2), what the convolution and pooling steps do and why (this part), and why the target is a logarithm. Nothing else is hidden inside this network.

80,833 parameters in total — small by deep-learning standards, because the graph representation already does most of the work.

PART FOUR

Our Data and Training

What to train on, and how three models beat one

What to train on

1,213 CIFs – Materials Project structures, no labels. The **prediction** set.

10,987 matbench crystals – DFT-computed K, G, the paper's own benchmark. The **training** set.

Only 278 of the 1,213 appear in matbench – tempting to label just those.

The structures were never the limit – the labels were

Intersecting an unlabelled collection with a labelled benchmark discards 97% of the labels that exist. Invert the roles: train on all 10,987 matbench crystals, treat the 1,213 CIFs purely as a prediction set – 40× more training data, and what the paper itself does.

When labels are the scarce resource, never let an unlabelled collection define the training set.

Three models, averaged

Two networks trained on the same data with different random initialisations land in different minima. They **agree** about the signal and **disagree** about their own noise — so averaging keeps the first and partly cancels the second.

Average in log space

10 and 1000 GPa average to 100 GPa in log space, 505 GPa linearly. 100 is the defensible answer.

One shared split

Initialisation varies between the three models; the train/val/test split does not — or the ensemble's score is measured partly on data some member trained on.

Free uncertainty

Spread between the three predictions is a per-crystal confidence signal — large spread means the ensemble is guessing.

Ensembling typically buys 10–15% lower error. Ours gained 9% on bulk modulus, in that range.

PART FIVE

Results

How well it actually works

Model performance

Model	MAE $\log_{10}$(GPa)	R^2	MAE (GPa)	Rel. error
Bulk, single model	0.0696	0.882	11.54	17.4%
▶ **Bulk, 3-model ensemble**	**0.0630**	**0.901**	**10.20**	**15.6%**
Shear, single model	0.0836	0.889	7.71	21.2%
▶ **Shear, 3-model ensemble**	**0.0781**	**0.899**	**7.27**	**19.7%**

Measured on 1,648 held-out crystals. All models share one train/validation/test split, so every row is directly comparable.

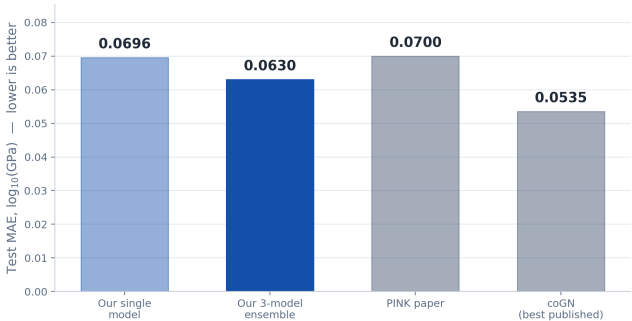
No overfitting

Train 0.0318 vs. test 0.0630 — ordinary generalisation error, not recall.

Ensembling gain

9% of the bulk-modulus error removed by averaging three initialisations.

How we compare to the literature



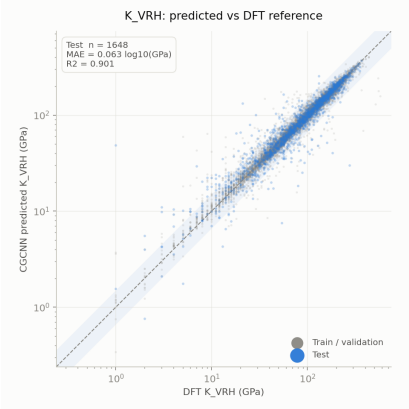
Reading this: our bulk ensemble slightly **beats** the paper it reproduces.

It sits between PINK and coGN – the best result ever published on this benchmark, with any architecture.

Same architecture, same training data as the paper. The gain comes from ensembling and a cosine learning-rate schedule.

Reproduction target met: 0.0630 against the paper's ≈0.070.

Predicted vs. DFT --- bulk modulus



Every point is one crystal: DFT value on *x*, our prediction on *y*. A perfect model puts every point on the diagonal.

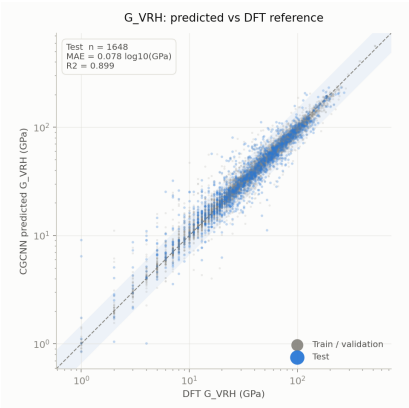
Points above the line are over-predictions; below, under-predictions.

The shaded band marks "within a factor of 1.5" — a useful screening tolerance, where ranking matters more than the exact value.

Blue = the 1,648 held-out test crystals.
Grey = training data, shown for context.

$R^2 = 0.901$ **— the model explains 0.901 of the variance in bulk modulus across the whole test set.**

Predicted vs. DFT --- shear modulus



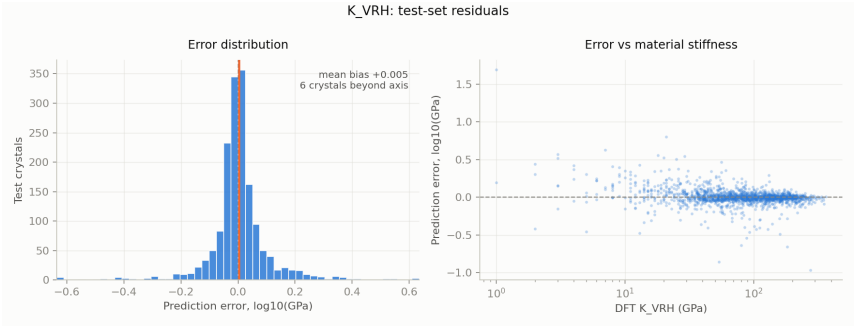
MAE 0.0781 against 0.0630 for bulk — consistently worse, for a physical reason.

Bulk modulus resists uniform compression, dominated by average bond stiffness — close to what a graph network reads off directly.

Shear modulus resists *shape change*, which depends on bond *directionality* and anisotropy the representation captures less directly.

The same ordering appears in the paper and across the matbench leaderboard — a property of the task, not of this implementation.

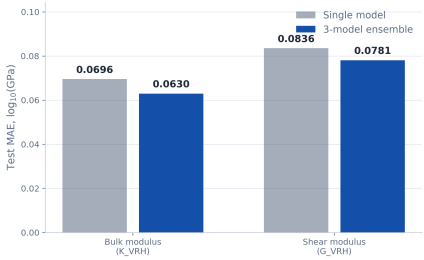
Where the errors live



Left: unbiased. The error distribution centres on zero — the model does not systematically over- or under-predict.

Right: the honest weakness. Errors fan out below ∼30 GPa — least reliable on soft materials, precisely the ultralow-κ regime PINK targets.

Ensembling, and how it trains



Consistent gains on both targets – about 9% of the error, in line with the 10–15% typically expected.

Cosine annealing over 200 epochs. Train and validation track closely throughout – no divergence, so no overfitting.

How low can the error go?

Because the model trains on $\log_{10}$, MAE converts to a **multiplicative** error of $10^{\text{MAE}} - 1$.

MAE 0.0630 → within a factor of 1.156 → 15.6% typical error

A ∼2% relative error is not reachable on this task – by anyone

2% needs MAE $= 0.0086$ – about six times better than the best result ever published on this benchmark, with any architecture. The blocker is **label noise**, not model capacity: the targets are DFT-computed moduli, and DFT elastic constants themselves disagree with experiment by roughly 5–15%. A model cannot be more accurate than the labels it is fitted to – one reporting 2% here would be revealing a leak, not an achievement.

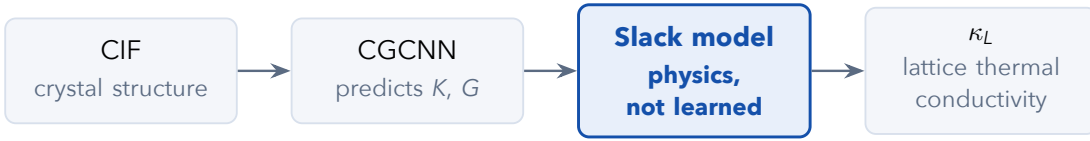
Realistic floor for this architecture: ≈0.06 $\log_{10}$, about 15%. We are at 0.0630.

PART SIX

From Moduli to Thermal Conductivity

The physics half of PINK: not learned, but just as important

Picking the pipeline back up



Parts Two through Five covered the first arrow only: a learned network turning a crystal graph into two numbers, K and G. Those numbers are not PINK's actual output – they are an *input* to a second stage that is not machine learning at all.

The Slack model is a closed-form formula from the phonon-transport literature, decades old and unchanged from the original paper. Given K, G, density, cell volume, and atomic mass, it returns κ_L directly – no training, no data, no gradient descent.

Everything from here on is arithmetic, not learning – the only open question is whether the arithmetic is implemented correctly.

The formula, ingredient by ingredient

Simplified to the terms that matter – the full expression lives in `scripts/07_predict_kappa.py`, checked line-for-line against the paper's own code (200 random cases, 10^{-9} relative tolerance):

$$\kappa_L \propto G \cdot v_{\mathbf{sound}} \cdot V^{1/3} \cdot e^{-\gamma} / N$$

Sound velocity $v_{\mathbf{sound}}$

Computed from K, G, and density – how fast a vibration travels through the lattice.

Grüneisen parameter γ

How *unevenly* the bonds stretch as the lattice vibrates. Enters as $e^{-\gamma}$ – the single most sensitive term in the formula.

G, $V^{1/3}$, N

Shear stiffness, a cell-size length scale, and atom count – each enters once; none is where this talk's story lives.

Unlike the other terms, γ is not a free input – it is derived purely from predicted K and G. Part Seven finds a place where that was not what happened.

How sure is the model? Monte Carlo uncertainty

The ensemble already gives every crystal a *K* and *G spread* — three models rarely agree exactly. Rather than average that spread away before the physics stage, it is propagated through.

2,000 draws per crystal from $\mathcal{N}(\text{prediction}, \text{spread})$ for $\log_{10} K$ and $\log_{10} G$ independently, each pushed through the identical Slack-model formula, summarised as a 5th–95th percentile interval.

Analogy: instead of one weather forecaster giving one number, run 2,000 slightly different forecasters — each starting from a plausible *K* and *G* — and report the *range* they land in, not just their average.

A single pre-trained model has no equivalent of this — one *K*, one *G*, one κ_L, no sense of how confident that number is.

Sanity check: do we agree with the paper's own model?

Real experimental κ_L is rare – the reason PINK predicts it at all. The cheapest check available first: run the paper's own pre-trained CGCNN through *our* physics implementation, on the same 1213 crystals, and ask whether two independently-trained models agree.

Pearson $r = 0.932$, Spearman $\rho = 0.926$, MAE $= 0.169$ $\log_{10}$ ($\approx$47.5% typical error). 10/20 overlap among each model's 20 lowest-κ_L picks.

Strong agreement – but this is still two models agreeing with each other, not yet a test against reality.

PART SEVEN

Does It Match Reality?

A real accuracy gap, and tracing it to its source

The real test: 45 materials with a measured κ_L

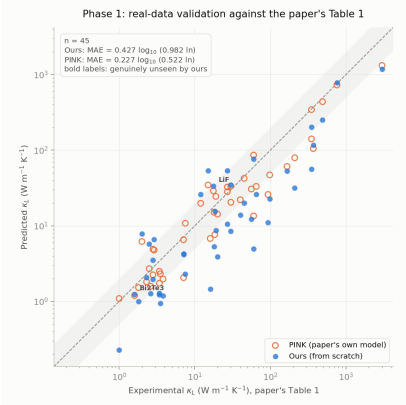
Everything so far compares predictions to *other predictions* — DFT labels, or the paper's own pretrained model. Table 1 of the paper is different: 46 real materials with κ_L actually measured in a lab. 45 have a Materials Project structure this project could fetch (GaP/mp-3490 is deprecated in MP's own database).

Why this table is special: the paper also tabulates *its own* predicted shear modulus, sound velocity, and Grüneisen parameter for every material — enough to localise a disagreement, not just report one.

Run the full pipeline — CGCNN ensemble → Slack model, nothing special-cased — on these 45 structures, and compare against both the experimental value and the paper's own prediction.

Real, independent, lab-measured data is the strongest test available. It is also where this reproduction's story gets interesting.

First look: we seem to lose



Told plainly, not softened: against these 45 real measurements, our from-scratch pipeline looks noticeably *worse* than the paper's own pretrained model.

MAE: ours **0.427** vs. paper's **0.227** $\log_{10}$

Head-to-head: paper closer on 37/45 materials, ours closer on only 8/45

Every earlier check in this talk said our moduli were fine. So where is this gap actually coming from?

Two suspects, cleared

κ_L is built from the ingredients in Part Six. Check each one against the paper's own tabulated value for these same 45 crystals, one at a time:

Shear modulus G – cleared

Pearson $r = 0.988$ against the paper's own predicted G, on these exact 45 crystals. Excellent agreement.

Sound velocity – cleared

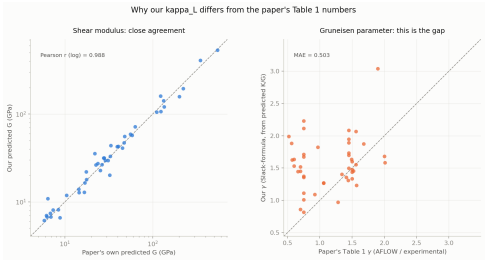
Folds in our bulk modulus K too. Pearson $r = 0.991$ against the paper's own value. Also excellent – K is fine.

Both moduli-derived quantities agree closely with the paper's own numbers. Whatever is causing the gap on the previous slide, it is not K or G.

Two suspects down, one ingredient left unchecked: the Grüneisen parameter γ – the exact term Part Six flagged as "remember this one."

Found it: the Grüneisen parameter

Right panel below: our γ against the paper's own tabulated γ, for the same 45 crystals.



Mean offset **+0.42** – our γ runs systematically *too high*, worst for covalent semiconductors (GaAs, InSb, ZnTe). Because $\kappa_L \propto e^{-\gamma}$, this alone explains the gap.

Not a bug in the moduli, not a bug in the formula – one specific input, systematically wrong.

Why: a number only a known material has

The paper's own Results text says it directly, for Table 1 specifically: *Grüneisen parameters were obtained from the AFLOW database and experimental data.* Real, looked-up values – not derived from predicted K and G the way every other number in this pipeline is.

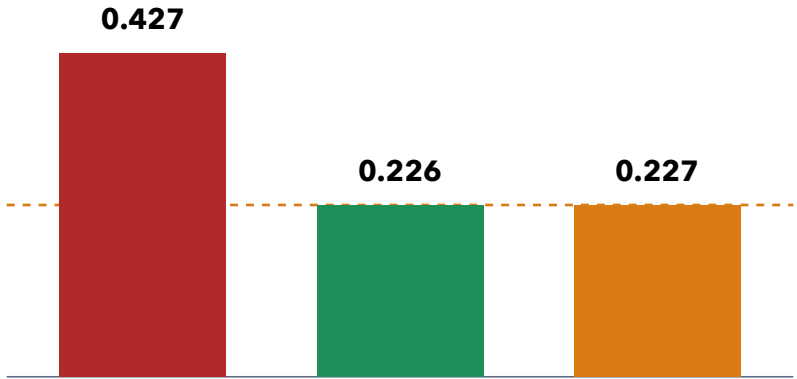
The catch: an AFLOW/experimental lookup only exists for a material someone has already synthesised and measured – a brand-new, never-synthesised GNoME candidate has no such number. Confirmed by inspection: the paper's own released `pink_predict.py` has no AFLOW lookup either; like this reproduction, it derives γ purely from K and G.

Table 1's headline accuracy, in other words, was quietly measured under conditions the paper's own actual use case can never provide.

A weather forecaster graded as excellent on days they were allowed to peek at tomorrow's newspaper.

The fix: same K, G, structure --- only γ swapped

Hold our predicted K, G, and structure exactly fixed. Replace only γ with the paper's own tabulated value, for these 45 crystals only – the one substitution a hypothetical GNoME candidate could never make.



Honest caveat: 43 of 45 already seen

One question the last four slides deliberately deferred: how many of these 45 materials did the CGCNN ensemble already see during training?

43

already in the matbench training set — recall, not generalisation

2

genuinely unseen — LiF and Bi_2Te_3

Both numbers reported separately throughout this section — never blended into one aggregate that overstates what 43-of-45 can prove.

On the 2 unseen materials specifically, ours and the paper's model are both within a factor of ∼2 of experiment — too small a sample to draw a strong conclusion from alone, but directionally consistent with everything above.

The γ-substitution result is a controlled comparison, not a generalisation claim — it shows which ingredient explains the gap, not that this pipeline is proven on unseen chemistry.

PART EIGHT

The Actual Screen

554,054 candidate materials, and what came out the other end

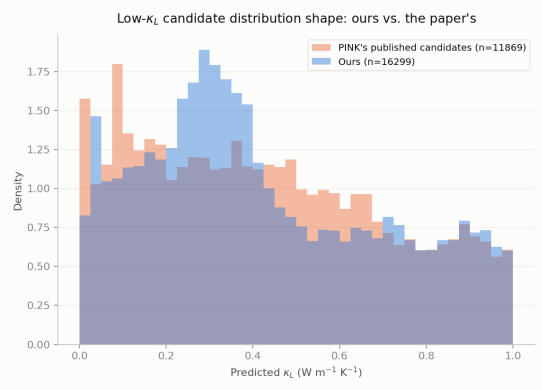
What GNoME is, and the funnel

Google DeepMind's GNoME is a public database of computationally-discovered stable materials — the paper's actual screening target, and the reason this pipeline exists at all. This project's download has **554,054** stable materials; the paper's own screen used **377,221** — GNoME's public data has grown since, so this is a current screen using the paper's stated criteria, not a bit-identical replay.

Stage	Ours (554,054)	Paper's (377,221)
Bandgap $[0.1, 3.0]$ eV + stable	37,889	30,199
+ no radioactive elements	33,323	26,305
+ $\kappa_L \leq 1$ (own model)	**16,299**	**11,869**

Every stage narrows by roughly the same proportion as the paper's own funnel — despite a $\sim$1.5$\times$ larger starting corpus and a fully independently-trained model.

25 minutes, one ordinary laptop



The full 33,323-candidate screen — CGCNN ensemble, Slack-model physics, Monte Carlo uncertainty, all of it — ran end to end in about **25 minutes** on an ordinary, several-year-old laptop CPU. No cluster and no GPU required for inference at this scale.

Both candidate-list shapes are similar and span the same range — ours somewhat more concentrated around 0.25–0.4 W m^{-1} K^{-1}.

The bottleneck this whole project started from — DFT being too slow to screen hundreds of thousands of materials — is exactly what this number answers.

Do we agree with the paper's own discoveries?

The paper ships its actual published candidate list. Match by reduced chemical formula (verified: same pymatgen convention on both sides, zero mismatches in a 200-row spot check) and ask: how much overlap is there, really?

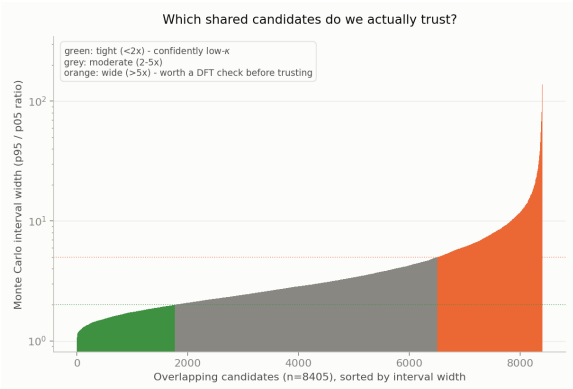
70.8%

of the paper's own 11,869 published candidates also appear in our independently-derived list (8,405 materials)

The other direction: 51.6% of our 16,299 candidates also appear in the paper's smaller, older list — the remainder are, in large part, candidates that simply did not exist yet in the paper's 377,221-material snapshot, not disagreement.

Two models sharing nothing but architecture family and physics formula, screening overlapping but non-identical corpora, converging on most of the same real discoveries — the strongest evidence in this

New: which candidates do we actually trust?



Every one of the 8,405 shared candidates carries a Monte Carlo κ_L interval from the ensemble's own *K/G* spread – something a point-estimate pipeline cannot produce at all.

1,770 confidently low-κ ($< 2\times$)
4,739 moderate confidence (2–5$\times$)
1,896 wide interval – worth a DFT check before trusting ($> 5\times$)

The paper's own pipeline can name a candidate. It cannot say which ones it is actually sure about – this can.

PART NINE

Wrapping Up

41 · CONTRIBUTION

What we did differently from the paper

1. **Written from scratch** – graph construction, convolution, training loop, physics: our own implementation throughout, no pre-trained weights ever loaded.
2. **Ensembles with propagated uncertainty** – three initialisations per target, worth 9% of the error, spread carried through as a Monte Carlo κ_L interval the paper lacks.
3. **Provenance on every prediction** – each row records whether the model trained on that crystal, so recall is never mistaken for generalisation.
4. **Diagnosed a real-data gap, then closed it** – traced Table 1's accuracy gap to the Grüneisen parameter's source and confirmed the diagnosis by substitution, rather than stopping at a worse number.
5. **Ran the actual screen** – all 554,054 current GNoME candidates, not a 1,213-crystal demonstration, converging on 70.8% of the paper's own published discoveries.

Deliberately identical where it should be: the architecture, the training data, the log target, the physics formula. In a reproduction, differences should be few and intentional.

The deliverables

results/pink_moduli_predictions.csv

Bulk and shear modulus for all 1,213 crystals – Pugh ratio, DFT reference where one exists, ensemble uncertainty, and a provenance tag.

935
unseen

278
matbench recall

42
held-out test

results/gnome_screen_candidates.csv + gnome_overlap.csv

16,299 scored GNoME candidates at $\kappa_L \leq 1$ with a Monte Carlo confidence interval each; 8,405 cross-referenced against the paper's own published list.

A built-in correctness check: on the 42 test-provenance crystals the error matches the benchmark test error exactly.

A second architecture: ALIGNN

CGCNN's convolution reads bond *lengths* only. ALIGNN adds a second graph over bond *angles* – geometry a plain bond graph cannot see. Same identical split, same 1,648-crystal test set:

Target	Model	MAE $\log_{10}$(GPa)	R^2
Bulk	CGCNN ensemble	0.0630	0.901
Bulk	**ALIGNN**	**0.0539**	**0.920**
Shear	CGCNN ensemble	0.0781	0.899
Shear	**ALIGNN**	**0.0725**	**0.901**

ALIGNN wins on both targets – both moduli depend on bond angles, information CGCNN structurally cannot see. One ALIGNN model beats a 3-model CGCNN ensemble – an unfavourable comparison, if anything.

Trained on Kaggle after Colab lost the session three times in one evening – an unattended kernel sidesteps that failure mode.

Closing the loop: ALIGNN's own κ_L

Same 1,213-crystal prediction set, same unchanged Slack-model physics – swap CGCNN's moduli for ALIGNN's and see if the final κ_L still agrees.

$$r = 0.940$$

Pearson correlation between ALIGNN's κ_L and our own CGCNN-ensemble κ_L

Spearman $\rho = 0.947$, MAE $= 0.141$ $\log_{10}$. 12/20 lowest-κ_L overlap.

Tighter than CGCNN's own agreement with the paper's pretrained model ($r = 0.932$).

Two architectures sharing only data, split, and physics converge this closely – more evidence for a real signal, not architecture noise.

Summary

- A crystal is a **graph**, read by a **gated convolution** stacked three times — shown concretely in real NaCl.
- **Ensembling** three models on all 10,987 matbench crystals landed at 0.0630 $\log_{10}$(GPa) for bulk modulus, matching and slightly beating the paper this work reproduces.
- **Physics, not more learning**, turns (K, G) into κ_L — bit-identical to the paper's own code, extended with Monte Carlo uncertainty it lacks.
- Real-data validation **looked worse at first** (0.427 vs. 0.227) — traced to the Grüneisen parameter, and **closed** by substitution (0.226).
- The actual **554,054-material screen** independently recovers 70.8% of the paper's own published discoveries.
- A second architecture, **ALIGNN**, trained on the identical split **beats the CGCNN ensemble on both moduli** — 0.0539 vs. 0.0630 $\log_{10}$ MAE for bulk.

Thank you.